

UNIVERSITÀ DEGLI STUDI DI NAPOLI FEDERICO II

Dipartimento di Ingegneria Elettrica e delle Tecnologie dell'Informazione (DIETI)

Corso di Ingegneria del Software – A.A. 2025/2026 – Canale 2

BugBoard26

Documentazione di Progetto

Specifica dei Requisiti, Design del Sistema e del Software, Testing

Autore: Marco Lerro – Matricola N86005403
Gruppo di lavoro: Gruppo singolo (1 componente)
Stack tecnologico: ASP.NET Core, React, PostgreSQL, Docker

16 luglio 2026

Indice

1	Introduzione	2
1.1	Scopo del documento	2
1.2	Panoramica del sistema BugBoard26	2
1.3	Funzionalità assegnate	2
1.4	Stack tecnologico	3
2	Specifica dei Requisiti Software	4
2.1	Glossario	4
2.2	Definizione degli utenti target	4
2.3	Modellazione dei casi d'uso	5
2.4	Requisiti non funzionali e di sistema	6
2.4.1	Requisiti non funzionali	6
2.4.2	Requisiti di sistema	6
2.5	Formalizzazione dei casi d'uso significativi	7
2.5.1	Caso d'uso: Creazione di una Issue	7
2.5.2	Caso d'uso: Ricerca di Issue tramite Parola Chiave	9
3	Design del Sistema	10
3.1	Architettura proposta	10
3.1.1	Criteri di design adottati	10
3.2	Scelte tecnologiche e motivazioni	10
3.2.1	Back-end: ASP.NET Core	10
3.2.2	Front-end: React	11
3.2.3	Persistenza: PostgreSQL	11
3.2.4	Containerizzazione: Docker	11
4	Design del Software	12
4.1	Schema di persistenza dei dati	12
4.2	Diagramma delle classi di design	12
4.3	Motivazione delle scelte di software design	12
4.4	Versioning	13
4.5	Report di qualità del codice	13
4.6	Codice sorgente e artefatti di build	13
5	Attività di Testing	15
5.1	Test plan	15
5.2	Test di unità automatici	15
5.3	Strategie di progettazione dei test	16
5.4	Valutazione dell'usabilità	16

1 Introduzione

1.1 Scopo del documento

Il documento costituisce la documentazione di progetto richiesta dal committente SoftEngUniNA, nell'ambito del corso di Ingegneria del Software A.A. 2025/2026. Il documento raccoglie, in un unico elaborato strutturato in capitoli, la specifica dei requisiti software, il design del sistema e del software, e la documentazione delle attività di testing relative alle funzionalità assegnate del sistema **BugBoard26**.

1.2 Panoramica del sistema BugBoard26

BugBoard26 è una piattaforma per la gestione collaborativa di issue in progetti software. Il sistema consente a team di sviluppo di segnalare problemi relativi a un progetto, monitorarne lo stato, assegnarli a membri del team e tenere traccia delle attività di risoluzione.

1.3 Funzionalità assegnate

Il committente ha assegnato il seguente sottoinsieme non negoziabile di funzionalità, tra quelle elencate nella Sezione 3.1 del documento dell'incarico:

ID	Descrizione sintetica
1	Sistema di autenticazione basato su email/password. Account amministratore precaricato con credenziali di default; gli amministratori possono creare ulteriori utenze (normali o di amministrazione).
2	Tutti gli utenti autenticati possono segnalare una issue (titolo, descrizione, priorità opzionale, allegato immagine opzionale). Le issue possono essere di tipo <i>question</i> , <i>bug</i> , <i>documentation</i> , <i>feature</i> e nascono nello stato <i>todo</i> .
3	Vista riepilogativa delle issue, con filtro e ordinamento per tipologia, stato, priorità e altri parametri rilevanti.
5	Sezione commenti associata a ciascuna issue, in cui tutti gli utenti possono lasciare messaggi, aggiornamenti o richieste di chiarimento.
9	Gli utenti possono modificare solo le issue a loro assegnate; gli amministratori hanno accesso completo in scrittura.
11	Funzione di ricerca che consente di trovare issue in base a parole chiave.
18	Gli amministratori possono impostare scadenze opzionali per la risoluzione di una issue.

Nota: le funzionalità non assegnate (es. assegnazione a membri del team con notifica, dashboard aggregata, export, etichette, cronologia modifiche, archiviazione, suggerimento automatico asse-

gnatario, modalità readonly, gestione duplicati, report mensili) sono escluse dal perimetro del presente progetto e non sono state implementate.

1.4 Stack tecnologico

Il sistema è realizzato come applicazione web distribuita, rispettando il vincolo di separazione netta tra back-end e front-end richiesto dalla commessa:

- **Back-end:** ASP.NET Core Web API (C#), che espone i servizi applicativi tramite API REST, completamente indipendente dal front-end.
- **Front-end:** applicazione web Single Page Application realizzata in React, che comunica con il back-end esclusivamente tramite le API REST esposte.
- **Persistenza:** PostgreSQL, gestito tramite Entity Framework Core come ORM.
- **Containerizzazione:** Docker e Docker Compose per la gestione dei container di back-end e database, a supporto della portabilità e del deployment.

Le motivazioni dettagliate delle scelte tecnologiche sono riportate nel Capitolo 3.

2 Specifica dei Requisiti Software

2.1 Glossario

Termine	Definizione
Issue	Elemento informativo che rappresenta una segnalazione all'interno del sistema (domanda, bug, problema di documentazione o richiesta di funzionalità).
Tipo di Issue	Categoria di una issue: <i>question, bug, documentation, feature</i> .
Stato	Fase del ciclo di vita di una issue (es. <i>todo, in progress, done</i>).
Priorità	Attributo opzionale di una issue che ne indica l'urgenza (es. bassa, media, alta).
Commento	Messaggio testuale associato a una issue, inserito da un utente autenticato.
Scadenza (deadline)	Data opzionale, impostabile solo da un amministratore, entro cui una issue dovrebbe essere risolta.
Utente Standard	Utente autenticato con permessi limitati: può creare issue, commentare, modificare solo le issue a lui assegnate.
Amministratore	Utente con permessi estesi: può creare utenze, ha accesso completo in scrittura a tutte le issue, può impostare scadenze.
Autenticazione	Processo di verifica dell'identità di un utente tramite coppia email/password.
Token di sessione (JWT)	Token firmato digitalmente utilizzato per mantenere lo stato di autenticazione tra front-end e back-end senza sessione stateful lato server.
API REST	Interfaccia di programmazione esposta dal back-end, basata sui principi architetturali REST, tramite cui il front-end accede ai dati e alle funzionalità.

2.2 Definizione degli utenti target

Il sistema individua tre categorie di attori:

- **Amministratore:** utente con responsabilità di gestione del sistema e del team. Crea le utenze, ha accesso in scrittura completo a tutte le issue e può impostarne le scadenze.
- **Utente Standard:** membro del team di sviluppo o segnalatore, che utilizza il sistema per segnalare problemi, consultarli, commentarli e aggiornare lo stato delle issue a lui assegnate.
- **Stakeholder:** utente che è limitato alla sola visualizzazione delle issue.

Tutte le categorie devono autenticarsi prima di poter accedere alle funzionalità del sistema; non sono previsti accessi anonimi.

2.3 Modellazione dei casi d'uso

La Figura 2.1 riporta il diagramma dei casi d'uso relativo alle funzionalità assegnate. Il diagramma è stato realizzato in Visual Paradigm; il file, come richiesto, è disponibile nel repository di progetto.

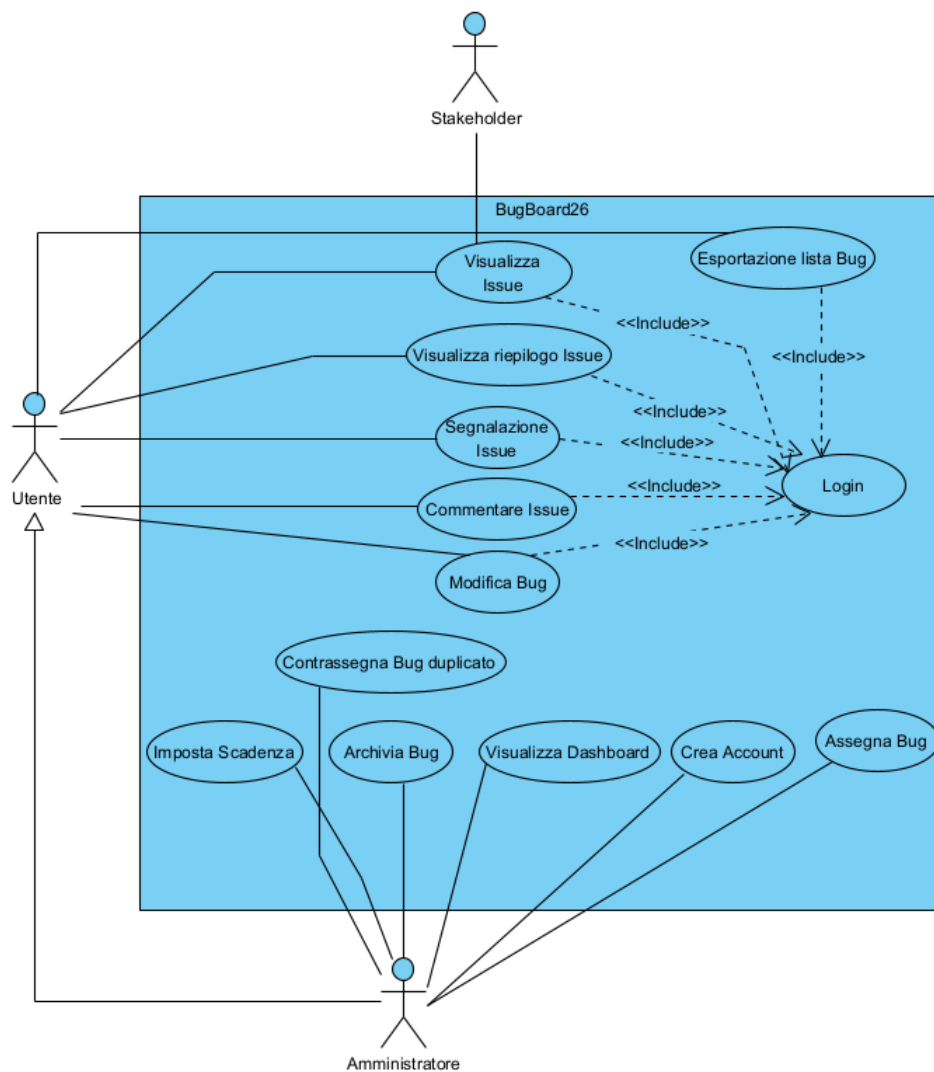


Figura 2.1: Diagramma dei casi d'uso (bozza) – funzionalità assegnate

2.4 Requisiti non funzionali e di sistema

2.4.1 Requisiti non funzionali

Categoria	Descrizione
Sicurezza	Le password sono memorizzate esclusivamente in forma hash; l'autenticazione avviene tramite token JWT firmato; tutte le comunicazioni client-server dovrebbero avvenire su HTTPS in ambiente di produzione.
Autorizzazione	Ogni endpoint del back-end applica controlli di autorizzazione basati sul ruolo (Amministratore/Utente Standard) e sulla proprietà della risorsa (es. una issue è modificabile da un utente standard solo se a lui assegnata).
Usabilità	L'interfaccia deve permettere di creare una issue e trovarne una tramite ricerca in un numero ridotto di passaggi (idealmente ≤ 3 click dalla schermata principale).
Affidabilità	Il sistema deve gestire correttamente input non validi restituendo messaggi di errore comprensibili, senza causare stati inconsistenti nel database.
Portabilità	L'intero back-end e il database devono poter essere eseguiti tramite container Docker, riproducibili su qualunque host che supporti Docker Engine.
Manutenibilità	L'architettura a livelli e l'uso di pattern di astrazione (Repository, DTO, Dependency Injection) devono consentire l'estensione futura con nuove funzionalità (es. quelle non assegnate nella traccia) senza modifiche invasive al codice esistente.
Performance	Le operazioni di lista/ricerca/filtro sulle issue devono restituire risposta in tempi ragionevoli anche al crescere del numero di record, tramite paginazione e query indicizzate lato database.

2.4.2 Requisiti di sistema

- Back-end: .NET 8 (o successivo), eseguito in container Docker Linux.
- Database: PostgreSQL 15+ (o successivo), eseguito in container Docker separato.
- Front-end: applicazione React (Node.js 18+ in fase di build), compatibile con i principali browser desktop aggiornati (Chrome, Firefox, Edge).
- Comunicazione: API REST su protocollo HTTP/HTTPS, formato dei payload JSON.

2.5 Formalizzazione dei casi d'uso significativi

Come richiesto, seguono due casi d'uso tramite template di A. Cockburn, seguiti da Mock-Up visuale delle interfacce coinvolte.

2.5.1 Caso d'uso: Creazione di una Issue

UC-01 – Creazione di una Issue	
Attore primario	Utente autenticato (Utente Standard o Amministratore)
Scopo	Consentire a un utente autenticato di segnalare un nuovo problema o richiesta al sistema
Livello	Obiettivo utente (user-goal)
Stakeholder e interessi	<i>Utente</i> : vuole registrare rapidamente una segnalazione. <i>Amministratore/Team</i> : vuole ricevere segnalazioni complete e correttamente categorizzate.
Precondizioni	L'utente ha effettuato l'accesso al sistema con credenziali valide.
Postcondizioni	Una nuova issue è persistita nel sistema con stato iniziale <i>todo</i> , associata all'utente segnalante e con timestamp di creazione registrato nella cronologia.
Trigger	L'utente seleziona l'azione "Nuova Issue" dall'interfaccia.
Scenario principale	1. L'utente seleziona "Nuova Issue". 2. Il sistema mostra il form di creazione (titolo, descrizione, tipo, priorità opzionale, allegato opzionale). 3. L'utente compila titolo, descrizione e seleziona il tipo (question/bug/documentation/feature). 4. L'utente conferma l'invio. 5. Il sistema valida i dati obbligatori (titolo e descrizione non vuoti). 6. Il sistema crea la issue con stato <i>todo</i>, la associa all'utente autenticato e la salva. 7. Il sistema mostra conferma e reindirizza alla vista di dettaglio della issue creata.
Estensioni	3a. L'utente specifica anche una priorità: il valore viene incluso nella issue. 3b. L'utente allega un'immagine: il sistema effettua l'upload e associa il riferimento al file alla issue. 5a. Titolo o descrizione mancanti: il sistema mostra un messaggio di errore e non salva la issue; si ritorna al passo 3. 5b. Formato o dimensione dell'allegato non valido: il sistema segnala l'errore e consente di riprovare senza perdere i dati già inseriti.
Requisiti speciali	Il campo tipo è obbligatorio con valore di default <i>question</i> se non selezionato esplicitamente.
Frequenza	Alta: è l'operazione più frequente svolta dagli utenti standard.

The image shows a mock-up of a 'Nuova issue' (New issue) form. The form is contained within a rounded rectangle. At the top, the title 'Nuova issue' is centered. Below the title, there are two main input fields: 'Titolo' (Title) and 'Descrizione' (Description). The 'Titolo' field is a single-line text input, and the 'Descrizione' field is a larger, multi-line text area. Below these fields, there are three optional fields: 'Tipo' (Type), 'Priorità (opz.)' (Priority), and 'Allegato (opz.)' (Attachment). Each of these fields has a dropdown menu with a placeholder text: '[scegli tipo ▼]', '[---- ▼]', and '[Scegli file....]' respectively. At the bottom of the form, there are two buttons: 'Annulla' (Cancel) on the left and 'Crea Issue' (Create Issue) on the right. The 'Crea Issue' button is highlighted in a teal color.

Figura 2.2: Mock-up (bozza) – Form di creazione Issue

2.5.2 Caso d’uso: Ricerca di Issue tramite Parola Chiave

UC-02 – Ricerca di Issue tramite Parola Chiave	
Attore primario	Utente autenticato (Utente Standard o Amministratore)
Scopo	Consentire all’utente di individuare rapidamente una o più issue in base a una parola chiave
Livello	Obiettivo utente (user-goal)
Stakeholder e interessi	Utente: vuole ritrovare rapidamente issue rilevanti senza scorrere l’intero elenco.
Precondizioni	L’utente ha effettuato l’accesso al sistema. Esiste almeno una issue nel sistema.
Garanzia di successo (postcondizioni)	Viene mostrato all’utente l’elenco delle issue il cui titolo e/o descrizione contengono la parola chiave inserita.
Trigger	L’utente digita un termine nella barra di ricerca.
Scenario principale	1. L’utente accede alla vista riepilogativa delle issue. 2. L’utente digita una parola chiave nel campo di ricerca. 3. Il sistema invia la richiesta di ricerca al back-end. 4. Il back-end interroga il database filtrando le issue per corrispondenza (parziale, case-insensitive) su titolo e descrizione. 5. Il sistema restituisce e mostra l’elenco delle issue corrispondenti, mantenendo eventuali filtri/ordinamenti già attivi.
Estensioni	5a. Nessuna issue corrisponde alla parola chiave: il sistema mostra un messaggio di “nessun risultato trovato”. 2a. L’utente cancella il testo di ricerca: il sistema ripristina l’elenco completo (con eventuali altri filtri attivi).
Requisiti speciali	La ricerca deve restituire risultati in tempo utile (percepito come istantaneo) anche con centinaia di issue in database, grazie a indicizzazione lato database.
Frequenza	Media-alta.

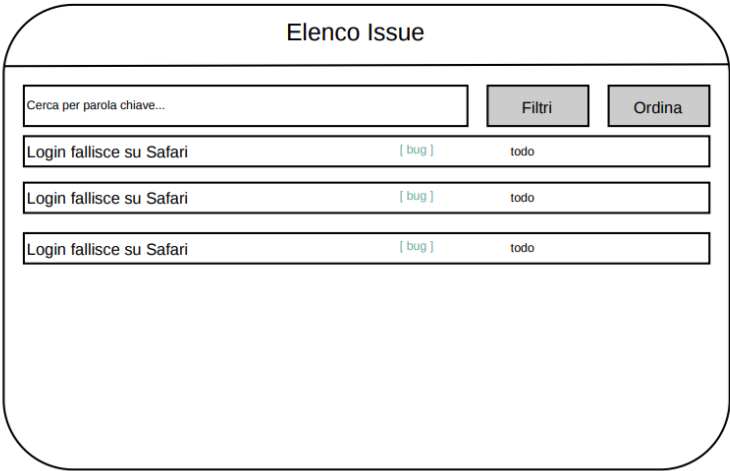


Figura 2.3: Mock-up (bozza) – Ricerca e Vista Riepilogativa delle Issue

3 Design del Sistema

3.1 Architettura proposta

Dato il vincolo imposto dalla commessa, il sistema è organizzato secondo un'architettura distribuita a due macro-componenti indipendenti: un back-end che espone API REST e un front-end SPA che si occupa esclusivamente dei servizi via rete. Il back-end è inoltre organizzato internamente secondo un'architettura a livelli (*layered architecture*), per favorire la separazione delle responsabilità e la manutenibilità.

3.1.1 Criteri di design adottati

- **Separazione back-end/front-end:** il front-end non ha alcuna conoscenza dell'implementazione interna del back-end e comunica esclusivamente tramite API REST versionate; questo consente di sostituire l'uno indipendentemente dall'altro.
- **Architettura a livelli nel back-end:** i controller REST (Presentation) delegano la logica di business ai Services (Application), che operano su Entità di Dominio tramite Repository (Persistenza), disaccoppiati dal dettaglio di accesso al database tramite interfacce.
- **Dependency Injection:** ASP.NET Core fornisce nativamente un container di Dependency Injection, utilizzato per iniettare Repository e Services, favorendo testabilità e sostituibilità dei componenti (es. con mock nei test di unità).
- **Data Transfer Object (DTO):** le entità di dominio non vengono mai esposte direttamente tramite le API; si utilizzano DTO dedicati per le richieste e le risposte, disaccoppiando il modello di persistenza dal contratto esposto al client.
- **Stato centralizzato lato server:** tutti i dati persistenti (issue, commenti, utenti, scadenze) sono gestiti unicamente dal back-end e dal database PostgreSQL; il front-end mantiene solo stato applicativo temporaneo (form, filtri correnti, token di sessione).

3.2 Scelte tecnologiche e motivazioni

3.2.1 Back-end: ASP.NET Core

ASP.NET Core è stato scelto per la realizzazione del back-end in quanto:

- è un framework Object-Oriented (C#) maturo, con tipizzazione statica che riduce gli errori a runtime;
- offre supporto nativo alla costruzione di API REST tramite i Web API Controller e il middleware pipeline;

- si integra nativamente con Entity Framework Core, un ORM completo per l'accesso a PostgreSQL, e con meccanismi di autenticazione basati su JWT;
- è multiplatforma e si presta naturalmente alla containerizzazione tramite immagini Docker ufficiali Microsoft.

3.2.2 Front-end: React

React è stato scelto per il front-end in quanto:

- consente di realizzare un'interfaccia a componenti riutilizzabili, adatta a una Single Page Application che utilizza esclusivamente API REST;
- il Virtual DOM garantisce buone performance di rendering anche con aggiornamenti frequenti dell'interfaccia (es. lista issue filtrata in tempo reale);
- l'ecosistema (React Router per la navigazione, Axios per le chiamate HTTP, eventuali librerie di state management) è ampio e maturo;
- l'architettura a componenti favorisce la completa indipendenza dal back-end, comunicando con esso solo tramite chiamate HTTP verso le API esposte.

3.2.3 Persistenza: PostgreSQL

PostgreSQL è stato scelto come sistema di gestione di basi di dati relazionale in quanto:

- garantisce proprietà ACID, fondamentali per la coerenza dei dati relativi a issue, assegnazioni e commenti;
- offre un ottimo supporto nell'ecosistema .NET tramite il provider Npgsql per Entity Framework Core;
- è open source, gratuito e ampiamente supportato dai principali provider di public cloud (Azure Database for PostgreSQL, AWS RDS);
- supporta indicizzazione full-text, utile per l'implementazione efficiente della funzionalità di ricerca per parola chiave (requisito 11).

3.2.4 Containerizzazione: Docker

Il back-end e il database sono containerizzati tramite Docker e orchestrati con Docker Compose. Questo garantisce:

- riproducibilità dell'ambiente di esecuzione indipendentemente dalla macchina host;
- possibilità di deployment su servizi di public cloud (es. Azure Container Apps, AWS ECS) in fase di discussione del progetto;
- isolamento tra il container applicativo e il container del database, ciascuno con il proprio ciclo di vita.

Nota: nessuna logica applicativa o di persistenza è delegata a servizi MBaaS esterni (es. Firebase, Supabase): tutta la logica di business risiede nel back-end ASP.NET Core e tutti i dati sono persistiti su un'istanza PostgreSQL controllata dal team di progetto, come espresso dal vincolo della commessa.

4 Design del Software

4.1 Schema di persistenza dei dati

Lo schema relazionale seguente riflette le funzionalità assegnate (1, 2, 3, 5, 9, 11, 18).

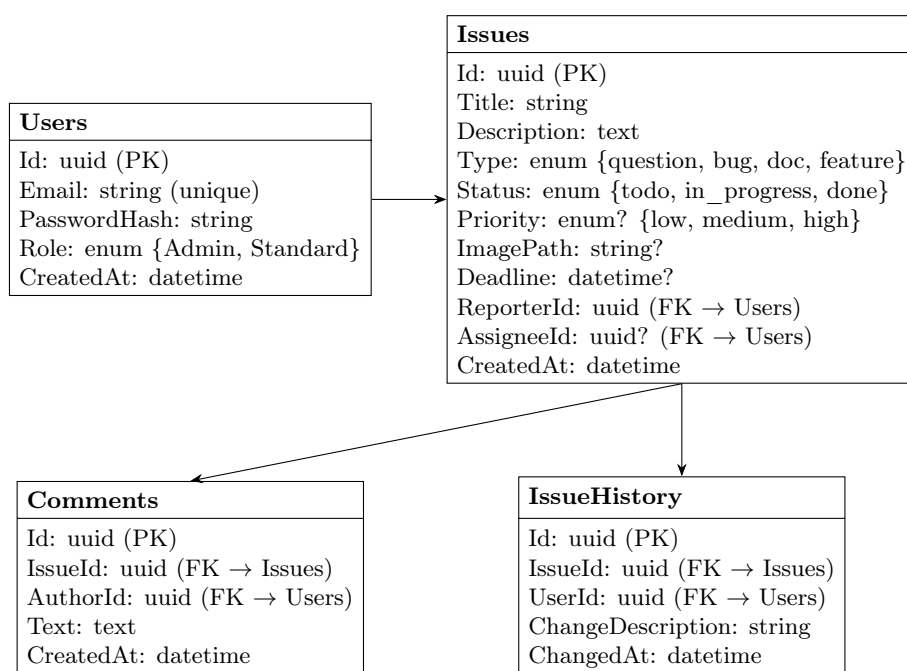


Figura 4.1: Schema concettuale della persistenza dati (bozza)

Nota implementativa: la tabella **IssueHistory** supporta il tracciamento delle modifiche (creazione, cambi di stato, riassegnazioni) ed è popolata automaticamente dal livello applicativo ad ogni operazione di scrittura rilevante sulle **Issues**, coerentemente con l'esigenza di tracciabilità richiamata nella specifica.

4.2 Diagramma delle classi di design

Il diagramma seguente mostra, in forma semplificata, le principali classi coinvolte nel back-end, secondo il pattern architetturale Controller–Service–Repository.

4.3 Motivazione delle scelte di software design

- **Repository Pattern:** astrae l'accesso ai dati dietro interfacce (**IIssueRepository**, **IUserRepository**), disaccoppiando i **Services** dal dettaglio di **Entity Framework Core** e facilitando la sostituzione con **mock** nei test di unità.

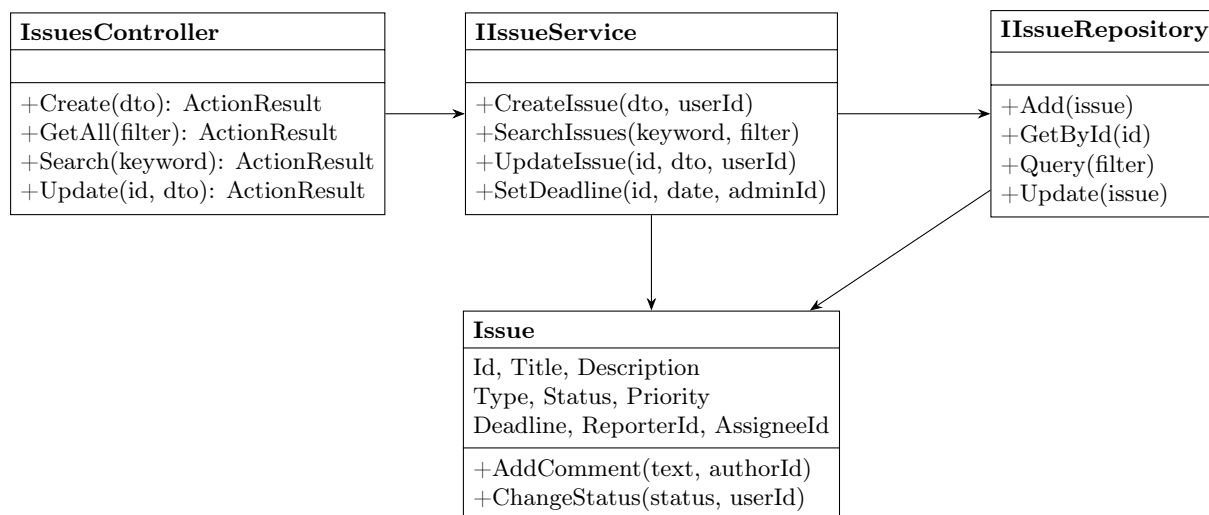


Figura 4.2: Diagramma delle classi di design (estratto, bozza)

- **Service Layer:** la logica di business (es. verifica che un utente possa modificare solo issue a lui assegnate – requisito 9 – o che solo un amministratore possa impostare scadenze – requisito 18) è centralizzata nei Services, non nei Controller, per favorire riuso e testabilità.
- **DTO e mapping:** l'uso di DTO per input/output disaccoppia il contratto API dal modello di dominio, permettendo di evolvere lo schema del database senza impatti diretti sul front-end.
- **Astrazione per riuso futuro:** le interfacce dei Repository e dei Services sono definite in modo generico rispetto all'entità gestita, così da poter essere estese in futuro alle funzionalità non assegnate nella presente iterazione (es. etichette, dashboard aggregata) senza dover riprogettare il livello di persistenza.

4.4 Versioning

Il codice sorgente del progetto è gestito tramite Git e ospitato su GitHub. <https://github.com/lerrooooo/bugboard26>

4.5 Report di qualità del codice

Il back-end è analizzato tramite SonarQube (o SonarCloud) per la verifica di code smell, duplicazioni e complessità ciclomatica.

- Report: *[inserire dopo sviluppo]*

4.6 Codice sorgente e artefatti di build

Il codice sorgente completo, comprensivo di Dockerfile per il back-end, docker-compose.yml per l'orchestrazione back-end/database, e file di build automatica, è disponibile nel repository GitHub indicato in Sezione 4.4.

Listing 4.1: Estratto indicativo di docker-compose.yml

```

1 services:
2   backend:
  
```

```
3     build: ./backend
4     ports:
5         - "5000:8080"
6     depends_on:
7         - db
8     environment:
9         - ConnectionStrings__Default=Host=db;Database=bugboard26;...
10 db:
11     image: postgres:16
12     environment:
13         - POSTGRES_DB=bugboard26
14         - POSTGRES_PASSWORD=[inserire]
15     volumes:
16         - dbdata:/var/lib/postgresql/data
17 volumes:
18     dbdata:
```

5 Attività di Testing

5.1 Test plan

Si riporta il test plan relativo alla funzionalità **Creazione di una Issue** (requisito 2).

Obiettivo	Verificare che la creazione di una issue avvenga correttamente con dati validi e sia correttamente rifiutata con dati non validi.		
Elementi da testare	Endpoint	POST /api/issues;	metodo
Casi di test	IssueService.CreateIssue. 1. Creazione con titolo e descrizione validi → issue creata con stato <i>todo</i>. 2. Creazione con titolo mancante → errore di validazione, nessuna issue creata. 3. Creazione con descrizione mancante → errore di validazione. 4. Creazione con priorità specificata → priorità correttamente salvata. 5. Creazione senza priorità → campo priorità nullo, nessun errore. 6. Creazione con tipo non valido (fuori enum) → errore 400. 7. Utente non autenticato tenta la creazione → risposta 401.		
Criteri di superamento	Tutti i casi di test producono l'esito atteso; nessuna issue parzialmente scritta in caso di errore.		

5.2 Test di unità automatici

Coerentemente con la modalità tradizionale (3 metodi non banali, con almeno due parametri ciascuno), sono stati sviluppati test automatici xUnit per i seguenti metodi del livello Service:

1. IssueService.CreateIssue(CreateIssueDto dto, Guid reporterId)
2. IssueService.UpdateIssue(Guid issueId, UpdateIssueDto dto, Guid requestingUserId)
– verifica anche la regola di autorizzazione del requisito 9
3. IssueService.SearchIssues(string keyword, IssueFilter filter)

Listing 5.1: Estratto di test xUnit per UpdateIssue

```
1 public class IssueServiceTests
2 {
3     private readonly Mock<IIssueRepository> _repoMock = new();
4     private readonly IssueService _sut;
```



```

5
6     public IssueServiceTests()
7     {
8         _sut = new IssueService(_repoMock.Object);
9     }
10
11     [Fact]
12     public async Task UpdateIssue_StandardUser_NotAssignee_ThrowsForbidden()
13     {
14         // Arrange
15         var issue = new Issue { Id = issueId, AssigneeId = otherUserId };
16         _repoMock.Setup(r => r.GetById(issueId)).ReturnsAsync(issue);
17         var dto = new UpdateIssueDto { Status = "in_progress" };
18
19         // Act & Assert
20         await Assert.ThrowsAsync<ForbiddenException>(
21             () => _sut.UpdateIssue(issueId, dto, currentUserId));
22     }
23
24     [Fact]
25     public async Task UpdateIssue_Admin_AnyIssue_Succeeds()
26     {
27         var issue = new Issue { Id = issueId, AssigneeId = otherUserId };
28         _repoMock.Setup(r => r.GetById(issueId)).ReturnsAsync(issue);
29         var dto = new UpdateIssueDto { Status = "done" };
30
31         var result = await _sut.UpdateIssue(issueId, dto, adminUserId);
32
33         Assert.Equal("done", result.Status);
34     }
35 }

```

5.3 Strategie di progettazione dei test

- **Classi di equivalenza:** per ciascun parametro sono state individuate classi di equivalenza valide/non valide (es. `keyword`: stringa vuota, stringa che matcha 0 risultati, stringa che matcha ≥ 1 risultato, stringa null).
- **Analisi dei valori limite (boundary testing):** per il parametro `filter` in `SearchIssues` sono testati i limiti di paginazione (pagina 0, pagina oltre l'ultima disponibile).
- **Test basati su ruolo:** per `UpdateIssue` sono stati definiti casi distinti per Utente Standard assegnatario, Utente Standard non assegnatario, Amministratore, coprendo la regola di autorizzazione del requisito 9.
- **Criteri di copertura strutturale:** obiettivo di copertura delle diramazioni (branch coverage) $\geq 80\%$ sui metodi testati, verificato tramite lo strumento di coverage integrato nella pipeline di test (`dotnet test -collect:"XPlat Code Coverage"`).
- **Isolamento tramite mock:** i Repository sono sostituiti con mock (Moq) nei test di unità dei Services, in modo da isolare la logica di business dal reale accesso al database.

5.4 Valutazione dell'usabilità

A complemento delle attività di testing funzionale, è stata condotta una valutazione preliminare dell'usabilità dell'interfaccia, secondo due modalità:

- **Ispezione basata su checklist:** revisione sistematica dell'interfaccia rispetto alle euristiche di Nielsen (visibilità dello stato del sistema, corrispondenza tra sistema e mondo reale, prevenzione degli errori, ecc.), applicata alle schermate di creazione issue e ricerca.
- **Test con utenti:** sessioni di osservazione con utenti reali/potenziati a cui viene chiesto di completare compiti rappresentativi (creare una issue, cercarne una per parola chiave, aggiungere un commento), seguite da un breve questionario di gradimento (es. System Usability Scale).